

■ e-Footwear Outlet

Aplicativo Android – Documentação Técnica

Disciplina:	Desenvolvimento Mobile – Android
Projeto:	Projeto Prático III
Repositório:	github.com/OFSilva-v2/e-commerce
Versão:	1.0.0
SDK Mínimo:	API 21 – Android 5.0 Lollipop
SDK Target:	API 34 – Android 14

1. Visão Geral do Projeto

O **e-Footwear Outlet App** é um aplicativo Android nativo desenvolvido em Java que encapsula o site de e-commerce de calçados e bolsas (disponível no repositório GitHub *OFSilva-v2/e-commerce*) dentro de uma WebView otimizada. O objetivo é proporcionar uma experiência mobile fluida para o catálogo de tênis, sapatos sociais e bolsas da loja.

Funcionalidades do site acessado:

- Catálogo com 12 produtos em 3 linhas (Tênis, Sapatos Sociais, Bolsas)
- Carrinho de compras funcional (cart.js)
- Sistema de checkout com múltiplas formas de pagamento
- Área do cliente com histórico de pedidos
- FAQ interativo e formulário de contato
- Design responsivo (adapta-se ao mobile)

2. Por que Desenvolver Aplicativos Android?

De acordo com **androidpro.com.br**, há razões sólidas para investir no desenvolvimento Android em 2024:

- **Domínio de Mercado:** Android detém aproximadamente 72% do mercado global de smartphones, representando bilhões de dispositivos ativos.
- **Base de Usuários:** A Google Play Store conta com mais de 2,5 bilhões de usuários ativos mensais em mais de 190 países.
- **Demanda Profissional:** Java e Kotlin para Android são habilidades altamente requisitadas pelo mercado de trabalho brasileiro e internacional.
- **Monetização:** O ecossistema oferece múltiplas fontes de receita: anúncios AdMob, compras in-app, assinaturas e vendas diretas.
- **Open Source & Flexibilidade:** Android é de código aberto, permitindo customizações profundas e integração com qualquer hardware ou serviço.
- **Ecossistema Rico:** Android Studio, Jetpack, Firebase, Google Maps API e centenas de bibliotecas facilitam o desenvolvimento de apps complexos.

3. Arquitetura e Estrutura do Projeto

3.1 Estrutura de Arquivos

Arquivo / Diretório	Descrição
AndroidManifest.xml	Permissões (INTERNET), declaração das Activities
SplashActivity.java	Tela de abertura animada com delay de 2,5s
MainActivity.java	Activity principal com WebView configurado
activity_splash.xml	Layout da splash: logo + gradiente + textos
activity_main.xml	Layout principal: Toolbar + WebView + SwipeRefresh
res/anim/	Animações XML (fade_in_scale, fade_in_slide)
res/drawable/	Vetores: logo de tênis, fundo degradê
res/menu/main_menu.xml	Menu de opções: Recarregar, Browser, Compartilhar
res/values/colors.xml	Paleta: azul-marinho, vermelho, dourado
res/values/themes.xml	Temas Material Design para o app
build.gradle (app)	Dependências, SDK targets, configurações de build
proguard-rules.pro	Regras de ofuscação para build Release

4. Componentes Técnicos Principais

4.1 WebView – Configurações Essenciais

O WebView é o componente central do app. Ele precisa ser configurado corretamente para renderizar o site de e-commerce com todas as funcionalidades JavaScript:

```
setJavaScriptEnabled(true) — Necessário para cart.js, main.js e products.js funcionarem
setDomStorageEnabled(true) — Habilita localStorage e sessionStorage do site
setUseWideViewPort(true) — Respeita a meta tag viewport do site responsivo
setLoadWithOverviewMode(true) — Ajusta o zoom inicial ao tamanho da tela
setCacheMode(LOAD_DEFAULT) — Usa cache para carregar mais rápido em 2ª visita
setBuiltInZoomControls(true) — Permite pinch-to-zoom para melhor usabilidade
```

4.2 WebViewClient – Tratamento de Navegação

A classe **WebViewClient** intercepta eventos de navegação. No app, é usada para:

- Exibir/ocultar a ProgressBar durante o carregamento da página

- Interceptar links tel: (ligar) e mailto: (enviar e-mail)
- Detectar erros de carregamento e exibir a tela de 'sem internet'
- Manter navegação interna no app (sem abrir o browser externo para o site)

4.3 SwipeRefreshLayout

Envolve o WebView para permitir o gesto de *puxar para atualizar* (pull-to-refresh), padrão familiar para usuários Android. As cores do indicador seguem a paleta da marca.

4.4 Verificação de Conectividade

O método **temConexao()** usa *NetworkCapabilities* (API 23+) para verificar se há Wi-Fi, dados móveis ou Ethernet ativos antes de tentar carregar o site. Caso não haja conexão, uma tela de erro amigável é exibida com botão 'Tentar novamente'.

5. Fluxo de Navegação do Usuário

Etapa	Activity / Ação	Comportamento
1	SplashActivity	Logo + animação fade-in por 2,5 segundos
2	→ MainActivity	Transição com fade suave entre telas
3	Verificação de Rede	Checa conexão antes de carregar URL
4a	WebView.loadUrl()	Carrega https://ofsilva-v2.github.io/e-commerce
4b	Sem Internet	Exibe tela de erro com botão 'Tentar novamente'
5	Navegação no site	Usuário navega livremente pelo e-commerce
6	Botão Voltar	WebView.goBack() navega no histórico da WebView
7	Menu de opções	Recarregar Abrir no browser Compartilhar Sobre

6. Dependências (build.gradle)

Dependência	Versão	Finalidade
appcompat	1.6.1	Compatibilidade com versões antigas do Android
material	1.11.0	Toolbar, MaterialButton, tema Material Design
swiperefreshlayout	1.1.0	Gesto pull-to-refresh no WebView
webkit	1.9.0	WebView avançado com melhorias de segurança
constraintlayout	2.1.4	Layouts flexíveis e responsivos

7. Como Importar no Android Studio

1. Abra o **Android Studio** (versão Hedgehog 2023.1+ recomendada)
2. Clique em **File** → **Open** e selecione a pasta *EFootwearApp/*
3. Aguarde o Gradle sincronizar as dependências (requer internet)
4. Conecte um dispositivo Android físico ou crie um AVD (emulador) no Device Manager
5. Clique em **Run 'app'** (Shift+F10) para compilar e instalar
6. O app abrirá com a tela de splash e em seguida carregará o site

■ Pré-requisito

O arquivo **local.properties** deve conter o caminho do SDK Android. Ele é gerado automaticamente pelo Android Studio ao abrir o projeto, ex: *sdk.dir=/Users/nome/Library/Android/sdk*. Esse arquivo não é versionado (.gitignore) pois é específico de cada máquina.

8. Permissões Declaradas no Manifesto

Permissão	Tipo	Motivo
INTERNET	Normal	Carregar o site no WebView
ACCESS_NETWORK_STATE	Normal	Verificar conectividade antes de carregar

Ambas são **permissões normais** (não perigosas) – não requerem aprovação do usuário em tempo de execução. Apenas são declaradas no `AndroidManifest.xml`.

9. Conceitos de Programação Demonstrados

- **Ciclo de Vida de Activities:** `onCreate`, `onPause`, `onResume`, `onDestroy` – gerenciamento correto de recursos do WebView em cada estado.
- **Handler e Runnable:** Temporizador não-bloqueante na `SplashActivity` usando `postDelayed()`.
- **Programação Orientada a Eventos:** `WebViewClient` e `WebChromeClient` como callbacks para eventos de carregamento.
- **Verificação de Rede Moderna:** Uso de `NetworkCapabilities` (API 23+) em vez do deprecated `NetworkInfo`.
- **Navegação Back Stack:** Interceptação do botão Voltar com `WebView.canGoBack()` / `goBack()`.
- **Internacionalização:** Strings centralizadas em `res/values/strings.xml`.
- **Material Design 3:** Uso de `MaterialButton`, `AppBarLayout` e `SwipeRefreshLayout`.
- **ProGuard:** Configuração de ofuscação e remoção de código para builds de produção.
- **Intents Implícitas:** Abertura de discador (tel:) e e-mail (mailto:) via `Intent.ACTION_DIAL/SENDTO`.

10. Sugestões de Melhorias Futuras

- Migração para Kotlin e Jetpack Compose (UI moderna declarativa)
- Notificações Push com Firebase Cloud Messaging (FCM) para promoções
- Cache offline com Service Worker ou Room Database para navegação sem internet
- Autenticação biométrica (impressão digital) para área do cliente
- Modo escuro (Dark Theme) com detecção automática da preferência do sistema
- Compartilhamento de produtos específicos via Deep Links
- Analytics com Firebase para rastrear produtos mais vistos

